Chess Blunder-Risk Analyzer: Technical Specification

Braden Hayes, Nate Poppe

CS5001 Senior Design

This document describes how we plan to build the system. It is based on version 1.1 of our Requirements Document (`Requirements_Document.md`), and all the REQ, NFR, and CON IDs used here are defined there.

1. Version and Change Log

Version	Date	Change	Author
0.1	10/29/26	First draft, put together from D0, D1, D2, and our project plan.	Nate Poppe
0.2	10/30/26	Added columns to the ImportJob table. Resuming (REQ-08) and recovering stuck imports (REQ-16) need the original request and the time of the last progress, and D1 didn't store them. Also gave each API failure case an HTTP status code.	Nate Poppe
1.0	10/30/26	Added the "estimated blunder risk" label to the Replay page for REQ-19 and CON-03.	Braden Hayes
1.1	10/30/26	Changes from our advisor review. The Model row of the technology stack now says we will compare linear regression against other model types. Updated to match Requirements Document 1.1.	Braden Hayes

2. Architecture Summary

Our system is a client-server application with a processing pipeline on the backend, as chosen in D0. The block diagram is in Section 2 of `documents/d0/d0-high-level-design.pdf`. It has six components:

- User Interface (Nate): a React and TypeScript website with Sign In, Import, Games, Replay, and Patterns pages. It only talks to our API.
- Game Import and Validation (Nate): gets games from Lichess or a PGN file, throws out unfinished or broken games, and handles resuming (D1 algorithm A1).
- Game and Results Storage (Nate): a PostgreSQL database for players, imports, games, moves, engine results, model versions, scores, and risk factors.
- Chess Feature Extraction (Braden): runs Stockfish over the player's moves at a fixed depth (D1 algorithm A2) and builds the model's features.
- Blunder-Risk Model (Braden): loads the active model version and scores moves in batches.
- Risk Explanation and Analytics (Braden): picks 2 or 3 risk factors per move and works out patterns across games.

D2 also added Lichess sign-in, a settings file checked at startup, and logging. The two outside systems are the Lichess API and Stockfish.

Everything runs on one computer with Docker Compose and three containers: the website, the FastAPI backend (which also runs Stockfish, the model, and the background job queue), and PostgreSQL. We don't use cloud services or a separate queue service (CON-01, and the D0 decision against microservices).

The data flow follows the D0 data-flow diagram and D2 Figures 2 and 3 (`documents/d2/d2-detailed-design.pdf`). The player signs in and starts an import. The backend checks the request, creates an import job, and works on it in the background. It pulls finished games newest first and saves each game with its moves in one transaction. Stockfish analyzes the position before each of the player's moves, the model scores them, and the explainer picks the risk factors, which are saved with the score. The Import page checks progress every second, and once the import is done the player can open the Games, Replay, and Patterns pages.

3. Interfaces and Data Formats

3.1 Component Boundaries

These are the D0 interfaces with the changes we made in D2.

ID	Between	Data	Format and protocol	Error behavior
I1	UI to Import	Import request or PGN file	JSON over HTTPS, file upload for PGN	422 for bad input, 409 if an import is already running, 503 if the engine or model is down
I2	Lichess to Import	Games, newest first	NDJSON over HTTPS	A 429 or dropped connection marks the import interrupted and keeps saved games
I3	Import to Storage	One checked game and its moves	Database rows, one transaction per game	Duplicates are skipped by the unique index (EC-07). Database down marks the import interrupted (EC-09).
I4	Storage to Feature Extraction	Player’s moves with position, clock, and game info	In-process Python call	Missing info uses the PR-02 fallbacks and is flagged
I5	Feature Extraction and Stockfish	Position in, best move and evaluations out	UCI over stdin/stdout	Crash or timeout restarts the engine and retries once, then marks the move failed (NFR-08)
I6	Feature Extraction to Model	One feature row per move	Python call, FeatureRow format	Features that don’t match the model’s list get the batch refused and logged (EC-16)
I7	Model to Explanation	Scores and feature contributions	Python call	No contributions means rule-based factors (EC-21). A failed move gets an empty score (EC-14).
I8	Explanation to Storage	One score and up to 3 factors	Database rows, one transaction	A failed write is retried once, then the game is marked partial
I9	Storage to UI	Games, moves, patterns, progress	JSON over HTTPS	Incomplete results show a status message. Another player’s data returns 404 (NFR-02).

3.2 API Endpoints

All endpoints start with `/v1` and use JSON, except the PGN upload. The player is identified by their session cookie, and every endpoint except sign-in and status returns 401 without one (REQ-01). Errors use one format:

```
        {"error": {"code": "import_running", "message": "You already have an import running.", "field":
null}}
```

Endpoint	Sends	Returns	Errors	Requirements
GET /v1/auth/login	Nothing	Redirect to Lichess	503 Lichess down	REQ-01
GET /v1/auth/callback	Code and state	Sets cookie, redirects to Import	400 cancelled or bad state	REQ-01, NFR-06
POST /v1/auth/logout	Nothing	204, clears cookie	None	REQ-01
GET /v1/status	Nothing	Engine and model status	None	REQ-15
GET /v1/lichess/players/{username}/summary	Time control	Number of matching games	404 not found, 502 Lichess down	REQ-03
POST /v1/imports	Username, time control, game count	202 with import job	422, 404, 409, 503	REQ-02, REQ-04
POST /v1/imports/pgn	A .pgn file	202 with import job, games read and skipped	413 over 5 MB, 422, 409, 503	REQ-05, REQ-17
GET /v1/imports/{id}	Import ID	Import job	404	REQ-06, NFR-02
POST /v1/imports/{id}/resume	Import ID	202 with import job	404, 409 not interrupted	REQ-08

Endpoint	Sends	Returns	Errors	Requirements
GET /v1/games	Limit, 1 to 50	Games newest first with top 3 moves	422	REQ-10
GET /v1/games/{id}/moves	Model version (optional)	Each move's position, score, best move, factors	404	REQ-09, REQ-11 to REQ-13
GET /v1/patterns	Nothing	Up to 5 patterns, partial flag	None	REQ-18

3.3 Message Formats

```
ImportJob   { job_id, source: "lichess" | "pgn", status: "running" | "interrupted" | "done",
              imported_n, target_n, analyzed_n, message }

GameMoves   { game_id, model_version,
              moves: [ { move_id, ply, san, fen_before, clock_ms, score (0-100 or null),
                         best_move, analysis_status: "ok" | "forced" | "failed",
                         factors: [ { rank, text, source: "model" | "rule" } ] } ] }

FeatureRow  { move_id, features: { name: value, ... },
              flags: { clock_available, rating_available, time_control_known } }
```

Braden will choose the feature names once the model type is decided (OQ-03). Each model version stores the feature list it expects, and every batch is checked against it.

3.4 Database Schema

This is our D1 data model (`documents/d1/d1-data-model.drawio`) with the D2 Figure 4 additions and the new ImportJob columns.

Table	Columns	Keys and rules
User	user_id, lichess_username	Username is unique
ImportJob	job_id, user_id, status, source, username, perf_type, target_n, imported_n, oldest_played_at, last_progress_at	One running import per player (EC-02)
Game	game_id, user_id, source_game_id, job_id, played_at, pgn	user_id plus source_game_id is unique (REQ-07)
Move	move_id, game_id, ply, fen_before, clock_ms, is_user_move	Belongs to a game
MoveAnalysis	move_id, best_move, eval_cp, status, features	Zero or one per move
ModelVersion	model_version_id, version_tag, is_active, model_file, feature_list, trained_at, metrics	Only one active version
RiskScore	move_id, model_version_id, game_id, score	Key is move plus model version (REQ-13)
RiskFactor	move_id, model_version_id, rank, text, source	Up to 3 per score

3.5 Lichess and Stockfish

We use `GET /api/user/{username}` to check the player exists and count their games, and `GET /api/games/user/{username}` with `ongoing=false` and `clocks=true` to stream games as NDJSON. On resume, `until` is set to the oldest saved game (OQ-06). We send one request at a time and stop on a 429 (CON-05). Sign-in uses Lichess OAuth, and we don't save the token (NFR-06).

We run several Stockfish processes through python-chess, up to one less than the number of CPU cores. Each one uses one thread, is reset before each game, and searches to a fixed depth so results are the same every run (NFR-01). Forced moves skip the engine.

3.6 Model and Explainer

Function	Returns	Fails when
load_active()	The active model and its version	The file is missing, so status reports the model down (EC-17)
predict(rows)	A 0 to 100 score per move (probability times 100)	The features don’t match, so the batch is refused (EC-16)
contributions(row)	How much each feature pushed the score	Not supported, so rule-based factors are used (EC-21)
explain(scored_moves)	Up to 3 factors per move from a phrase table	Never fails as a whole. Bad factors are dropped (EC-20).

An import keeps the model version it started with (EC-15). Comparing a new version (REQ-14) reuses saved features, so Stockfish doesn’t run again. The Replay page labels scores “estimated blunder risk” with a one-line note that it’s a probability (REQ-19).

4. Technology Stack

Layer	Choice	Why
Frontend	React and TypeScript	Interactive replay board and live progress (REQ-06, REQ-10, REQ-11)
Backend	Python, FastAPI, Pydantic	Input checks that name the bad field (REQ-02). Same language as the model (CON-01).
Sign-in	Lichess OAuth with signed cookies	No passwords to store (REQ-01, NFR-06)
Database	PostgreSQL with SQLAlchemy and Alembic	Unique keys enforce no duplicates and version-linked scores (REQ-07, REQ-13)
Chess rules	python-chess	PGN parsing and legal moves (REQ-05, REQ-09). GPL-3.0 (CON-06).
Engine	Stockfish	Best free engine (CON-01). Fixed depth gives repeatable results (NFR-01).
Model	Not chosen yet. Linear regression is our baseline, compared against other types such as logistic regression and tree-based models (OQ-03).	Must run on a CPU within 180 seconds (CON-01, NFR-04)
Deployment	Docker Compose	One command on one machine (NFR-03, CON-01)

5. Known Constraints

ID	Constraint	Rules out	Forces
CON-01	No budget	Paid engines, cloud hosting, GPU models	Free tools, one local machine, a small model
CON-02	Finished games only	Live scoring and browser overlays	Status check before saving (REQ-04) and `ongoing=false` on every request
CON-03	Score is a probability	Showing the score as a verdict	The “estimated blunder risk” label (REQ-19)
CON-04	Chess judgment from Braden	Treating factor wording as purely technical	Braden owns the features, blunder definition, and phrase table
CON-05	Lichess rate limits	Parallel requests and retry loops	One request at a time, resume on 429 (REQ-08)
CON-06	GPL-3.0 libraries	Closed-source release	GPL-compatible license for our code

6. Traceability Matrix

Test case IDs are the ones we plan to use in our Week 10 test plan.

Req ID	Description	Design Component (D0/D1/D2)	Status	Test Case
REQ-01	Lichess sign-in	D2 Authentication; EC-26 to EC-29	Designed	TC-01
REQ-02	Import request checks	D0 I1; D1 POST /v1/imports; EC-01	Designed	TC-02
REQ-03	Available game count	D2 summary endpoint	Designed	TC-03
REQ-04	Finished games only	D1 A1; EC-05; D2 Figure 2	Designed	TC-04
REQ-05	PGN upload	D1 POST /v1/imports/pgn; EC-03	Designed	TC-05
REQ-06	Import progress	D1 GET /v1/imports/{id}; D2 Import page	Designed	TC-06
REQ-07	No duplicate games	D1 Game unique index; EC-07	Designed	TC-07
REQ-08	Resume interrupted imports	D0 I2; D1 A1; EC-04; D2 Figure 2	Designed	TC-08
REQ-09	Score each move	D0 Blunder-Risk Model; D2 predict; EC-10	In progress	TC-09
REQ-10	Top 3 moves per game	D2 Games page; D1 RiskScore index	Designed	TC-10
REQ-11	Replay position and best move	D1 Move, MoveAnalysis; D2 Replay page	Designed	TC-11
REQ-12	Short risk factors	D2 explain, RiskFactor; EC-18, EC-20	Designed	TC-12
REQ-13	Scores tied to model versions	D1 ModelVersion, RiskScore; EC-08	Designed	TC-13
REQ-14	Compare model versions	D2 Model behavior step 6	Designed	TC-14
REQ-15	Engine and model status	D2 GET /v1/status; EC-17, EC-33	Designed	TC-15
REQ-16	Stuck import recovery	D2 PR-01; EC-38	Designed	TC-16
REQ-17	Missing PGN data fallbacks	D2 PR-02; EC-12, EC-39	Designed	TC-17
REQ-18	Player patterns	D2 GET /v1/patterns; EC-19	Designed	TC-18
REQ-19	Score labeled as an estimate	D2 Replay page (label added here)	Planned	TC-19
NFR-01	Same results every run	D1 A2 fixed depth; EC-11	Designed	TC-20
NFR-02	Player data isolation	D1 User to Game; D2 Authentication	Designed	TC-21
NFR-03	Runs on a regular computer	D0 hardware section; EC-30	In progress	TC-22
NFR-04	20 games in 180 seconds	D1 A2 parallel engines	In progress	TC-23
NFR-05	Fast read requests	D1 Game and RiskScore indexes	Designed	TC-24
NFR-06	Session and secret handling	D2 Authentication, Logging; EC-36	Designed	TC-25
NFR-07	First-time user task	D2 UI pages	Designed	TC-26
NFR-08	Engine failure isolated	D0 I5; EC-13; D2 Figure 3	Designed	TC-27
NFR-09	Error logging	D2 Logging; EC-34, EC-37	Designed	TC-28
CON-01	No budget	D0 decision log	Designed	TC-29
CON-02	Finished games only	D0 data flow; D1 A1	Designed	TC-30
CON-03	Score is a probability	D0 decision log; Replay label	Planned	TC-31
CON-04	Chess judgment from Braden	D0 component owners	Designed	TC-32
CON-05	Lichess rate limits	D0 I2; D1 A1	Designed	TC-33
CON-06	GPL-3.0 libraries	D1 build vs. reuse table	Designed	TC-34

There are no orphaned rows. REQ-19 and CON-03 had no design element in D0 through D2, so we added the label in Section 3.6. REQ-09 is in progress until the model type is chosen, and NFR-03 and NFR-04 are in progress until we time the system.

7. Advisor Review

What we discussed:

- Our plan had been to build the model with linear regression. Our advisor encouraged us not to commit to one approach this early and to explore other model types as well.

What we're doing because of it:

- Braden will treat linear regression as a baseline and compare it against at least one other model type before picking the final model (project plan T1 and T7).
- When comparing models we will also look at whether the output works as a probability (CON-03, OQ-04) and whether the model can report which features mattered, which decides if we get model-based risk factors (REQ-12).

Changes this review produced:

Change	**IDs affected**	**Logged in**
Linear regression is a baseline, not the final choice	OQ-03	Requirements Document 1.1
Updated the Model row of the technology stack	OQ-03, NFR-04	Technical Specification 1.1